

第七章

(第3、4、6题)

3. 假定一个虚拟存储系统的虚拟地址为 40 位，物理地址为 36 位，页大小为 16KB。若页表中有有效位、访问权限位、修改位、使用位，共占 4 位，磁盘地址不记录在页表中，则该存储系统中每个进程的页表大小为多少？如果按计算出来的实际大小构建页表，则会出现什么问题？

参考答案：

因为每页大小有 16KB，所以页内偏移地址占 14 位，虚页数为 $2^{40}B/16KB=2^{(40-14)}=2^{26}$ 页。

物理页面和虚拟页面大小相等，所以物理页号（页框号）的位数为 $36-14=22$ 位。

页表项位数为：有效位+保护位+修改位+使用位+物理页号位数= $4+22=26$ 位。

为简化页表访问，每项大小取 32 位。因此，每个进程的页表大小为： $2^{26} \times 32b=256MB$ 。

如果按实际计算出的页表大小构建页表，则页表过大（大于一个页面），从而无法按页处理页表，比如无法一次装入 256MB 大小的页表。

4. 假定一个计算机系统有一个 TLB 和一个 L1 Data Cache。该系统按字节编址，虚拟地址 16 位，物理地址 12 位，页大小为 128B；TLB 采用 4 路组相联方式，共有 16 个页表项；L1 Data Cache 采用直接映射方式，块大小为 4B，共 16 行。在系统运行到某一时刻时，TLB、页表和 L1 Data Cache 中的部分内容如下：

组号 标记 页框号 有效位 组号 标记 页框号 有效位 组号 标记 页框号 有效位 组号 标记 页框号 有效位

0	—	0	0	0	1	0	—	0	0	0	1
3			9	D		0			7	2	
1	2	1	0	—	0	0	—	0	0	—	0
3	D		2			4			A		
0	—	0	0	—	0	0	—	0	0	—	0
2			8			6			3		
0	—	0	6	0	1	0	3	1	7	—	0
7			3	D		A	4		2		

(a) TLB (4 路组相联)：4 组、16 个页表项

虚页号	页框号	有效位	行索引	标记	有效位	字节 3	字节 2	字节 1	字节 0
0	08	1	0	19	1	12	56	C9	AC
0									
0	03	1	1	—	0	—	—	—	—
1									
0	14	1	2	1B	1	03	45	12	CD
2									
0	02	1	3	—	0	—	—	—	—
3									
0	—	0	4	32	1	23	34	C2	2A
4									
0	16	1	5	0D	1	46	67	23	3D
5									
0	—	0	6	—	0	—	—	—	—

6		
0	07	1
7		
0	13	1
8		
0	17	1
9		
0	09	1
A		
0	–	0
B		
0	19	1
C		
0	–	0
D		
0	11	1
E		
0	0D	1
F		

7	16	1	12	54	65	DC
8	24	1	23	62	12	3A
9	–	0	–	–	–	–
A	2D	1	43	62	23	C3
B	–	0	–	–	–	–
C	12	1	76	83	21	35
D	16	1	A3	F4	23	11
E	33	1	2D	4A	45	55
F	–	0	–	–	–	–

(b) 部分页表：（开始 16 项）

(c) L1 Data Cache：直接映射，共 16 行，块大小为 4B

请问（假定图中数据都为十六进制形式）：

- （1）虚拟地址中哪几位表示虚拟页号？哪几位表示页内偏移量？虚拟页号中哪几位表示 TLB 标记？哪几位表示 TLB 组索引？
- （2）物理地址中哪几位表示物理页号？哪几位表示页内偏移量？
- （3）主存物理地址如何划分成标记字段、行索引字段和块内地址字段？
- （4）CPU 从虚拟地址 067AH 中取出的值为多少？说明 CPU 读取地址 067AH 中内容的过程。

参考答案：

- （1）16 位虚拟地址中低 7 位为页内偏移地址，高 9 位为虚页号；虚页号中高 7 位为 TLB 标记，低 2 位为 TLB 组索引。
- （2）12 位物理地址中低 7 位为页内偏移地址，高 5 位为物理页号（页框号）。
- （3）12 位物理（主存）地址中，低 2 位为块内地址，中间 4 位为 cache 行索引，高 6 位为标记。
- （4）地址 067AH=0000 0110 0111 1010B，所以，虚页号为 0000011 00B，映射到 TLB 的第 00 组，将 0000011B=03H 与 TLB 第 0 组的四个标记比较，虽然和其中一个相等，但对应的有效位为 0，其余都不等，所以 TLB 缺失，需要访问主存中的慢表。直接查看 0000011 00B =00CH 处的页表项，有效位为 1，取出物理页号 19H=11001B，和页内偏移 111 1010B 拼接成物理地址：11001 111 1010B=110011 1110 10。根据中间 4 位 1110 直接找到 cache 第 14 行(第 E 行)，有效位为 1，且标记为 33H=110011B，正好等于物理地址高 6 位，故命中。根据物理地址最低两位 10，取出字节 2 中的内容 4AH=01001010B。若读取的是小端方式机器中的 short 型数据，则 CPU 读出的数据为 2D4AH=00101101 01000101B。

6. 假设在 IA-32+Linux 平台上运行一个 C 语言源程序 P 对应的用户进程，P 中有一条循环语句 S 如下：

```
for (i=0; i<N; i++) sum+=a[i];
```

已知变量 sum 和数组 a 都是 long 型，链接后确定 a 的首地址为 0x8049300。假设编译器将 a 的首地址分配在 EDX 中，数组的下标变量 i 分配在 ECX 中，sum 分配在 EAX 中，赋值语句“sum+=a[i];”仅用一条指

令I实现，指令I的地址为0x8048c08。已知IA-32/Linux平台采用图7.14所示的两级页表分页虚拟存储管理方式，页大小为4KB，系统启动后控制寄存器CR0中的控制位NW和CD均为0。假定系统中没有其他用户进程，该进程的页目录表首地址为0x3d000，指令I对应页表项所在页表的首地址为0x5c8000，指令I所在页在主存中分配的页框号为1020，回答下列问题或完成下列任务。

- (1) 假定常数N在EBX中，手工写出循环语句S对应的指令序列，与GCC生成的目标代码进行比较。
- (2) 在执行到程序P时，控制寄存器CR0中的控制位PE和PG各是什么？
- (3) 指令I对应的汇编形式（AT&T格式）是什么？指令I中存储器操作数的寻址方式是哪种？
- (4) 若执行指令I时，CS段寄存器对应的描述符cache中存放的是表6.2中所示的用户代码段信息且CPL=3，DS段寄存器对应的描述符cache中存放的是表6.2中所示的用户数据段信息，则当i=50时，取指令操作过程中MMU得到的指令的线性地址是多少？取数操作过程中MMU得到的操作数的线性地址是多少？指令I所在页的虚页号是什么？指令I的线性地址中，页目录索引、页表索引和页内偏移量分别是什么？指令I对应的页目录项和页表项分别在主存哪个单元？指令I在主存哪个单元？第一次执行指令I时，指令I对应页表项中，字段P、R/W、U/S、A和D的内容各是什么？
- (5) 指令I在第一次执行过程中，有没有可能发生缺页异常？为什么？如果发生缺页异常的话，则页故障线性地址是什么？该地址会保存在哪个控制寄存器中？
- (6) 指令I在第一次执行过程中，有没有可能发生TLB缺失？为什么？若指令TLB共有16个表项，采用4路组相联方式，则虚拟页号中哪几位为TLB标记？哪几位表示TLB组索引？若第一次执行到指令I时，指令TLB中的部分内容如下表（TLB表项中部分字段缺失，内容以十六进制表示），则指令I所存放的主存地址是什么？

组号 标记 页框号 有效位 标记 页框号 有效位 标记 页框号 有效位 标记 页框号 有效位

0	0	1	0	0	1	0	0	0	0	0	0
3010	0A10		2101	D001		0000	0000		0000	0000	
0	0	1	0	0	1	0	0	1	0	0	0
2011	2D02		2012	28B0		4001	2012		0000	0000	
0	0	1	0	0	0	0	0	0	0	0	1
2001	8902		2120	9200		2010	340A		2301	320A	
0	0	1	0	0	0	0	0	1	0	0	0
1002	8770		0000	0000		A001	2010		0000	0000	

- (7) 若指令cache的数据区容量为8KB，主存块大小为32B，采用2路组相联映射方式，则指令I在第一次执行时的取指令过程中，会不会发生cache缺失？指令I所在的主存块应映射到指令cache的哪一组中？
- (8) 当N=2000时，数组a占用几个页面？每个页的虚页号是什么？数组元素a[1200]在哪个页中？

参考答案：

- (1) 假定常数N在EBX中，则以下指令序列可以实现循环语句S的功能：

```

movl    $0,%ecx
.LOOP:
cmpl    %ebx,%ecx
jge     .EXIT
addl    (%edx,%ecx,4),%eax
incl    %ecx
jmp     .LOOP

```

.EXIT

(2) 在执行到程序P时，已经是保护模式并采用分页虚拟管理方式，因此，控制寄存器CR0中的控制位PE=1（表示保护模式），PG=1（表示启用分页）。

(3) 因为IA-32中的long型和int型一样，都是32位带符号整数，因此，指令I对应的汇编形式(AT&T格式)是“addl(%edx,%ecx,4),%eax”。其中存储器操作数的寻址方式是“基址+比例变址+偏移量”，这里的偏移量为0，比例因子是4，因为在32位机器中每个数组元素(long型)的大小为4个字节。

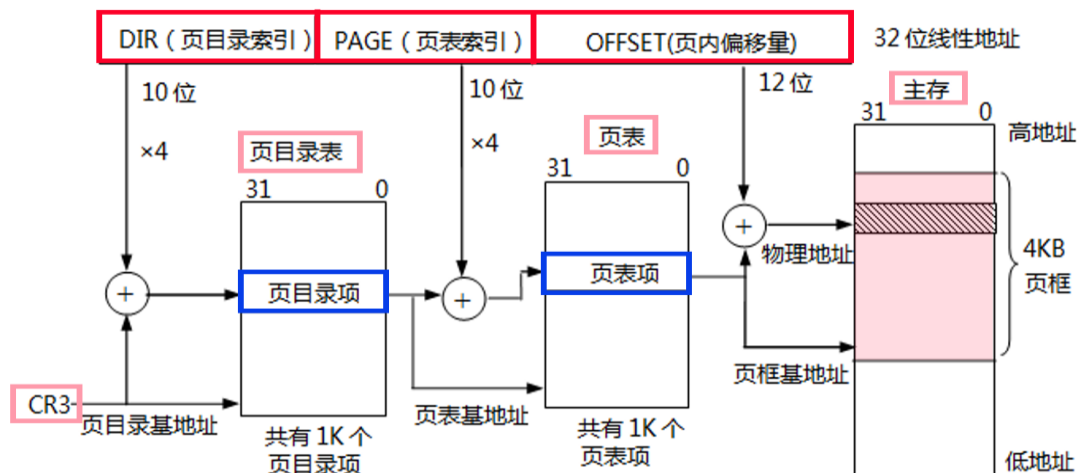
(4) 取指令操作过程中，MMU根据CS段寄存器对应的描述符cache中的信息（因为执行的是用户程序，所以用Linux初始化时设定的用户代码段信息）进行逻辑地址到线性地址的转换。MMU根据CS对应段描述符中的DPL，确定DPL的特权级别不高于CPL才能继续进行地址转换，这里DPL=CPL=3，没有发生存储保护错。因此，MMU将CS对应段描述符中的基地址与指令逻辑地址（指令逻辑地址为指令在代码段的段内偏移量）相加，得到线性地址为0x0+0x8048c08=0x8048c08。

取数操作中，MMU先根据DS对应段描述符获得DPL，确定DPL的特权级别不高于CPL才能继续进行地址转换，否则发生存储保护错。显然，这里DPL=CPL=3，没有发生存储保护错，于是，MMU将DS对应段描述符中的基地址与操作数有效地址相加得到线性地址。因为操作数“(%edx, %ecx, 4)”的寻址方式为“基址加比例变址加偏移量”，故有效地址EA=R[edx]+R[ecx]×4+0=0x80493300+50×4=0x80493c8。因此，操作数的线性地址为0x0+0x80493c8=0x80493c8。

指令I的线性地址为0x8048c08，其中低12位（1100 0000 1000）为页内偏移量，高20位（0000 1000 0000 0100 1000）为虚页号。虚页号中高10位（0000 1000 00）为页目录索引，低10位（00 0100 1000）为页表索引。由题干可知，该进程的页目录表首地址为0x3d000，指令I对应页表项所在页表的首地址为0x5c8000，指令I所在页在主存中分配的页框号为1020=0xffc，因此，指令I所在页对应的页目录项的主存地址为0x3d000+0000 1000 00<<2=0x3d080；指令I所在页对应页表项的主存地址为0x5c8000+00 0100 1000<<2=0x5c8120；指令I的主存地址为0xffc <<12+0xc08=0x00ff cc08。

线性地址空间划分： $4\text{GB} = 1\text{K个子空间} * 1\text{K个页/子空间} * 4\text{KB/页}$

- 页目录项和页表项格式一样，有32位（4B）



第一次执行指令I时，指令I所在页对应页表项中，P=1（所在页已在主存），R/W=0（所在页只能读和执行，不能写），U/S=1（所在页允许用户进程访问），A=1（所在页已被访问过），D=0（所在页为代码页，不会被修改）。

(5) 指令I在第一次执行过程中,取指令时不会发生缺页异常,因为,指令I不在一个页面起始处,在执行指令I前面的指令而发生缺页时,会将指令I一起调入内存。但是,取操作数a[0]时可能发生缺页,因为它所在的页面很有可能以前未被访问过,若是这样的话,执行指令I的过程中,当访问操作数时会发生缺页异常。此时,页故障线性地址是0x8049300,该地址保存在控制寄存器CR2中。

(6) 指令I在第一次执行过程中, 取指令时不会发生TLB缺失, 因为指令I不在一个页面起始处, 在

执行指令I前面的指令而发生TLB缺失时，已经将所在页的页表项装入TLB。但是，取操作数a[0]时有可能发生TLB缺失，因为a[0]所在页可能是第一次被访问，因而对应页表项可能不在TLB中。

已知指令TLB共有16个表项，采用4路组相联方式，因此，20位虚拟页号中高18位为TLB标记，低两位为TLB组索引。指令I的线性地址为0x8048c08，虚页号为0000 1000 0000 0100 1000，其中，TLB组索引为00，因此，MMU将TLB标记0000 1000 0000 0100 10（02012H）与第0组所有标记相比，在TLB的第0组中不存在一个标记相等且有效位为1的页表项，因而TLB不命中。

若TLB组索引（组号）为01，则可在第1组中找到一个标记为0000 1000 0000 0100 10（02012H）且有效位为1的页表项，此时TLB命中，可取出对应页表项中的页框号028B0H，从而得到对应的主存地址为0D001C08H=0xd001c08。

（7）已知指令cache的数据区容量为8KB，主存块大小为32B，因此，指令cache共有8KB/32B=256行。因为采用2路组相联映射，因此，cache组数为256/2=128，主存地址划分为：高20位为标记、中间7位为组索引、最低5位为块内地址。因为页大小为4KB，所以线性地址中低12位的页内地址与主存地址低12位相同，在MMU进行逻辑地址到物理地址转换的同时，CPU可以利用这低12位进行cache访问。

指令I的线性地址为0x8048c08，其中低12位（1100 0000 1000）为页内偏移地址，因此，组索引为110 0000，块内地址为01000。显然，指令I不在一个主存块的起始位置，因而，即使在第一次执行指令I时，在取指令阶段也不会发生cache缺失（在取它前面的指令时已经顺带把它装入了指令cache）。

指令I所在的主存块应映射到指令cache的第110 0000组中，即cache组号为96。

（8）当N=2000时，数组a占用空间大小为4×2000=8000字节，因为链接后a的首地址为0x8049300，而且对应段基地址为0，所以数组a的起始线性地址为0x8049300。因为4096-300=3796，8000-3796=4204>4096，因而数组a占了3个页面，分别含3796B、4096B和108B，虚页号分别是0000 1000 0000 0100 1001（0x8049）、0000 1000 0000 0100 1010（0x804a）、0000 1000 0000 0100 1011（0x804b）。因为4×1200=4800，而3796<4800<3796+4096，所以数组元素a[1200]在中间一个页面中，其虚页号为0x804a。

a[1200] 首 地 址 为 0x8049300+1200<<2=0x8049300+100 1011 0000B<<2=0x8049300+0x12c0=0x804a5c0。